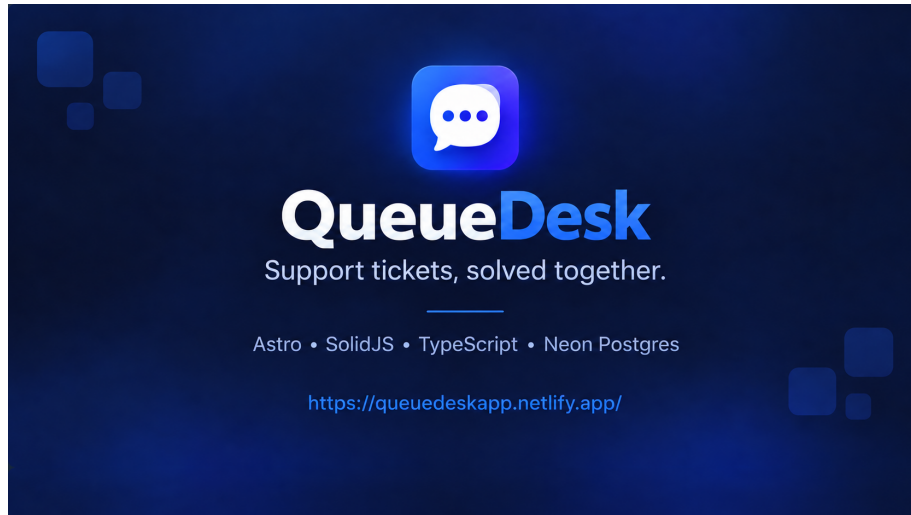




QueueDesk Project Specification v2

Support tickets, solved together.

Implementation profile: Astro + SolidJS + TypeScript + Neon Postgres + Netlify

QueueDesk is a compact helpdesk application for small service teams. Requesters submit support tickets and follow replies, while agents manage assignment, priority, status, internal notes and resolution. The implementation should emphasize clear authorization boundaries, auditable workflow transitions and a fast server-rendered UI.

Purpose of this document

This specification translates the product PRD into an implementation plan. It defines the application architecture, database shape, server interfaces, UI modules, service integrations, testing strategy, deployment model and delivery criteria.

1. Product and Functional Scope

User roles

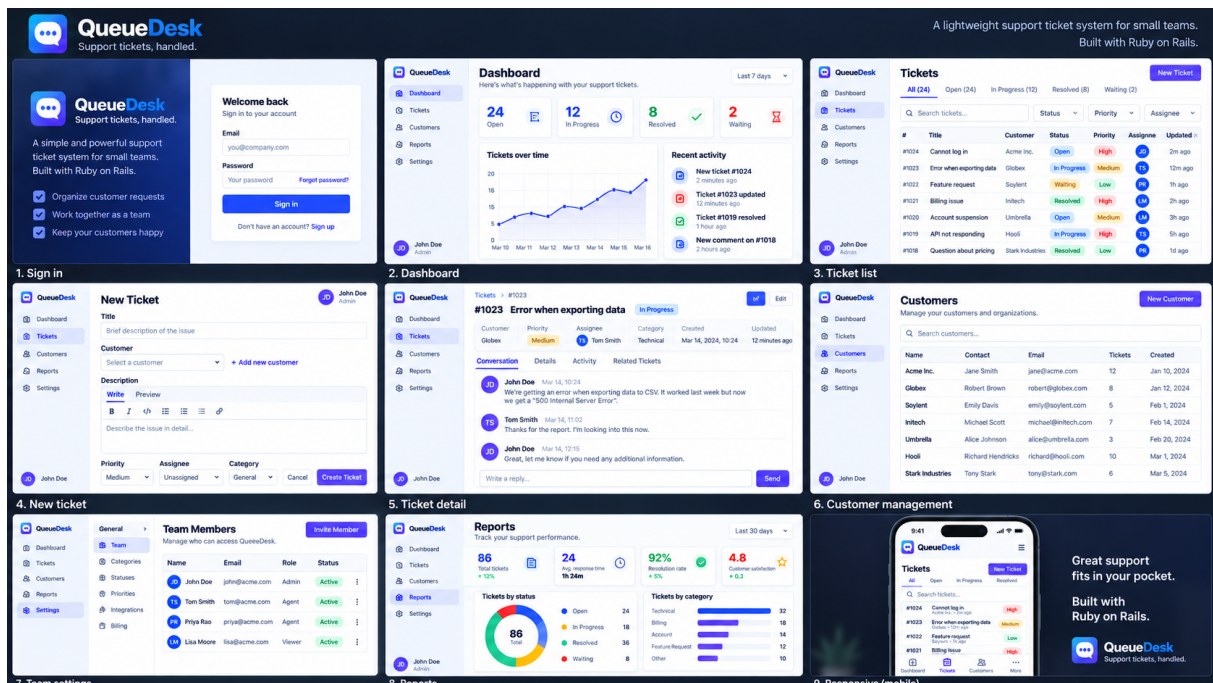
Requester	Create tickets, view own tickets, add public replies, upload attachments and reopen resolved tickets.
Support Agent	View team queue, assign tickets, change priority/status, add public replies or internal notes, resolve tickets.
Administrator	Manage users, categories, workspace settings and reports.

Required screens

- Sign in
- Dashboard
- Ticket queue
- New ticket
- Ticket detail
- Requester ticket list
- Team management
- Category management
- Reports
- Settings
- Mobile ticket list

Scope discipline

Build the primary workflow first. Optional analytics, advanced filters and secondary settings should only be added after the core workflow, permissions, tests and deployment are stable.



2. System Architecture

Target architecture

Browser -> Cloudflare DNS -> Netlify -> Astro server routes/pages -> SolidJS islands -> TypeScript service layer -> parameterized SQL -> Neon Postgres. Neon Auth provides authentication. R2, Resend and other services are added only where the project requires them.

Architecture principles

- Use Astro for routing, SSR, API/server endpoints and page composition.
- Use SolidJS only for interaction-heavy islands rather than turning the product into an unnecessary SPA.
- Keep server-only code outside the client bundle.
- Use direct parameterized SQL; centralize queries by domain to avoid SQL scattered through UI components.
- Validate every external mutation with Zod on the server.
- Authorization is enforced server-side even when controls are hidden in the UI.
- Keep infrastructure minimal; do not add services without a concrete product need.

Repository structure

```
src/ components/ layouts/ pages/ pages/api/ lib/ auth/ db/ validation/ services/ features/ styles/ tests/  
unit/ e2e/ public/ netlify/ functions/ (only where needed) .github/workflows/
```

3. Technology Stack

Application framework	Astro
Interactive UI	SolidJS
Language	TypeScript
Build tooling	Vite (through Astro)
Runtime	Node.js
Styling	Tailwind CSS
Database	Neon Postgres
Authentication	Neon Auth
Database access	Direct parameterized SQL - no ORM
Validation	Zod
Object storage	Cloudflare R2 where files are required
Email	Resend where transactional mail is required
Testing	Vitest + Playwright
Code quality	ESLint + Prettier
Monitoring	Sentry
CI/CD	GitHub Actions
Hosting	Netlify
DNS / edge	Cloudflare DNS
TLS	Netlify-managed certificate; Cloudflare proxy optional with Full (strict)

Node.js usage

Node.js is the server/runtime and tooling environment. Pin a current LTS version in the repository so local development, CI and Netlify use a consistent runtime.

Direct SQL conventions

- Use the Neon serverless/Postgres driver appropriate for the chosen Astro runtime.
- All values are passed as parameters; never construct SQL with untrusted string interpolation.
- Keep migrations as versioned SQL files in the repository.
- Wrap multi-step state changes in database transactions.
- Create indexes for foreign keys and fields used heavily by filtering/search.

4. Data Model

Core tables

users	id, email, display_name, role, created_at, updated_at
tickets	id, requester_id, assignee_id, category_id, title, description, status, priority, resolved_at, created_at, updated_at
ticket_comments	id, ticket_id, user_id, body, visibility, created_at
ticket_categories	id, name, description, active
ticket_attachments	id, ticket_id, comment_id, r2_key, filename, content_type, byte_size, uploaded_by
ticket_activities	id, ticket_id, user_id, action, metadata_json, created_at

Migration strategy

Store ordered SQL migration files in source control. Each migration must be forward-only for the portfolio MVP. Seed scripts should create realistic demo data without embedding production credentials.

Data integrity

- Use NOT NULL constraints for required fields.
- Use foreign keys for relational ownership.
- Use CHECK constraints for stable enums/statuses where practical.
- Use unique constraints for natural keys such as API keys, feature keys or emails where applicable.
- Use timestampz for timestamps.

5. Server Routes and UI Modules

Primary server interfaces

- GET /tickets - queue or requester-owned list
- POST /tickets - create ticket
- GET /tickets/:id - ticket detail
- POST /tickets/:id/comments - add public/internal comment
- POST /tickets/:id/assign - assign agent
- POST /tickets/:id/transition - status transition
- POST /api/uploads/presign - create R2 presigned upload request
- GET /reports - reporting dashboard

Primary UI components / Solid islands

- TicketTable
- TicketFilters
- TicketStatusBadge
- PriorityBadge
- ConversationThread
- ReplyComposer
- ActivityTimeline
- AttachmentList
- TicketSummaryCards

Implementation pattern

Pages should render useful content on the server. Solid islands handle stateful widgets such as filters, forms, calendars, progress displays, inspectors and interactive tables. Server mutations should return structured success/error results that map cleanly to form states.

6. Business Rules, Security and Integrations

Project-specific rules

- Requesters can only access tickets they own.
- Internal comments must never be returned to requester-facing queries.
- Every status, priority and assignment change creates an immutable activity event.
- Allowed status transitions are validated server-side.
- Attachment metadata is stored in Postgres; binary files live in R2.
- User-supplied rich text is escaped or sanitized before rendering.

External services

Neon Auth	Session and user identity.
Cloudflare R2	Ticket and comment attachments.
Resend	Created, assigned, replied and resolved notifications.
Sentry	Server and client exception monitoring.

Security baseline

- Rely on Neon Auth/session state server-side for identity.
- Validate authorization on every protected server route.
- Use Zod for request payload validation.
- Use secure, HTTP-only cookies according to the Neon Auth integration.
- Rate-limit abuse-sensitive public endpoints where practical.
- Never expose database, R2, Resend or Sentry server secrets in browser code.
- Escape/sanitize user-controlled text before rendering.
- Log errors to Sentry without logging secrets or sensitive payloads.

7. Testing and Quality

- Vitest: transition rules, authorization helpers, Zod schemas, filters/search helpers.
- Playwright: requester creates ticket; agent assigns/replies/resolves; requester reply; unauthorized cross-ticket access is denied.
- Security regression: internal notes never appear in requester response payloads.

CI pipeline

On each pull request / push: 1. npm ci 2. TypeScript typecheck 3. ESLint 4. Prettier check 5. Vitest 6. Build Astro application 7. Playwright on selected branches or against preview deployment

Quality gate

A feature is not considered complete until its server-side validation, authorization path and primary failure case have automated coverage.

8. Deployment and Operations

Netlify

- Connect the GitHub repository to Netlify.
- Use deploy previews for pull requests.
- Configure Astro's Netlify adapter/runtime as required by the final app mode.
- Store all secrets in Netlify environment variables.
- Use Netlify Functions/Background Functions only where the product requires them.
- Expose a lightweight health route where useful.

Cloudflare DNS / TLS

Point the chosen custom domain/subdomain to Netlify through Cloudflare DNS. Netlify can provision the origin/site certificate. If Cloudflare proxying is enabled, use Full (strict) so Cloudflare validates the Netlify certificate.

Environment variables

- DATABASE_URL
- NEON_AUTH_*
- R2_ACCOUNT_ID
- R2_ACCESS_KEY_ID
- R2_SECRET_ACCESS_KEY
- R2_BUCKET
- RESEND_API_KEY
- SENTRY_DSN

Observability

Configure Sentry for production and preview environments. Capture unhandled server errors and meaningful client failures. Add release/environment tags so a deployed error can be traced to a commit.

9. Implementation Sequence

- 1. Bootstrap repository: Astro, Solid, TypeScript, Tailwind, ESLint, Prettier, Vitest and Playwright.
- 2. Configure Neon Postgres and versioned SQL migrations.
- 3. Integrate Neon Auth and implement authorization helpers.
- 4. Build the core data model and seed/demo data.
- 5. Implement one complete end-to-end workflow before secondary screens.
- 6. Add project-specific external services such as R2 or Resend.
- 7. Add Solid islands only where interaction materially benefits from them.
- 8. Add Vitest coverage for domain logic and Playwright coverage for primary user journeys.
- 9. Configure GitHub Actions, Netlify previews and Sentry.
- 10. Polish README, screenshots, demo account/data and known limitations.

Definition of Done

- Requester can create and follow a ticket.
- Agent can assign, reply and resolve.
- Internal/public comment separation is correct.
- R2 attachments and Resend email work.
- Activity history is complete.
- Vitest and Playwright pass in CI.
- Netlify deployment is healthy.

Portfolio requirement

The repository README should include the project banner/logo, one-sentence value proposition, live demo link, screenshots, feature list, architecture summary, tech stack, local setup, environment-variable template, test commands, known limitations and license.